

範例 11-3 用 Migrations 建立資料庫及樣本資料

以下帶您走一遍 EF Core Migrations 使用方式，從建立 Model、DbContext、樣本資料、資料庫連線設定、在 DI Container 註冊 DbContext，最後再產生出對映的資料庫，請參考 Mvc7_Migrations 專案。

step 01 在 Models 資料夾建立 Employee 及 Register 資料模型

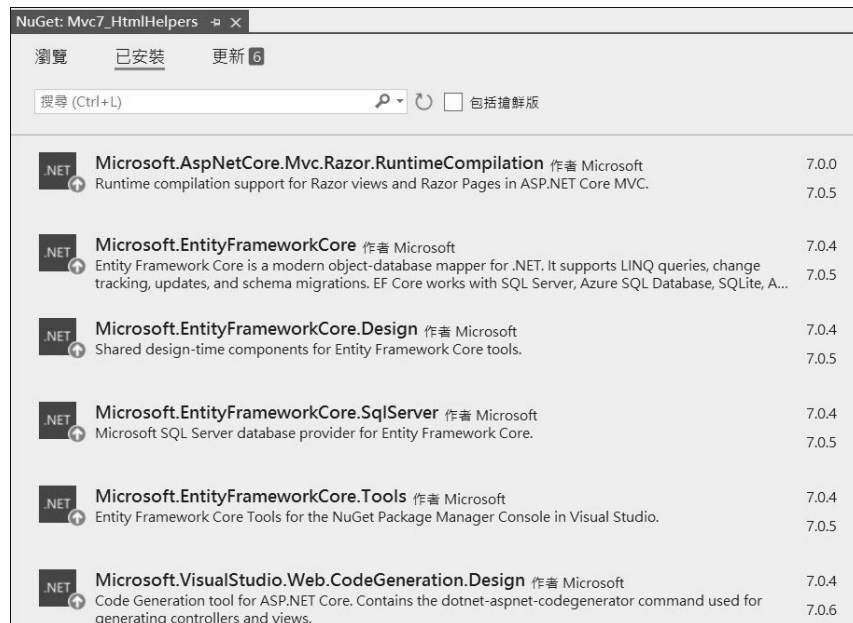
Models/Employee.cs

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Mobile { get; set; }
    public string Email { get; set; }
    public string Department { get; set; }
    public string Title { get; set; }
}
```

Models/Register.cs

```
public class Register
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Nickname { get; set; }
    public string Password { get; set; }
    public string Email { get; set; }
    public int Gender { get; set; }
    public int City { get; set; }
    public string Commutermode { get; set; }
    public string Comment { get; set; }
    public bool Terms { get; set; }
}
```

step 02 以【NuGet 套件管理員】安裝下圖 EF Core 與 SqlServer 等相關套件



或在命令視窗用 CLI 命令安裝：

```
dotnet add package Microsoft.EntityFrameworkCore --version 7.0.4
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 7.0.4
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 7.0.4
dotnet add package Microsoft.EntityFrameworkCore.Design --version 7.0.4
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design --version 7.0.4

dotnet list package (顯示專案安裝的套件)
```

step 03 在 Data 資料夾建立 CmsContext 類別及種子資料

 Data/CmsContext.cs

```
public class CmsContext : DbContext
{
    public CmsContext(DbContextOptions<CmsContext> options):base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }
    public DbSet<Register> Registers { get; set; }

    //建立樣本資料
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
    }
```

```

        modelBuilder.Entity<Employee>().HasData(
            new Employee { Id = 1, Name = "David", Mobile = "0935-155222",
                Email = "david@gmail.com", Department = "總經理室", Title = "CEO" },
            new Employee { Id = 2, Name = "Mary", Mobile = "0938-456889",
                Email = "mary@gmail.com", Department = "人事部", Title = "管理師" },
            new Employee { Id = 3, Name = "Joe", Mobile = "0925-331225",
                Email = "joe@gmail.com", Department = "財務部", Title = "經理" },
            new Employee { Id = 4, Name = "Mark", Mobile = "0935-863991",
                Email = "mark@gmail.com", Department = "業務部", Title = "業務員" },
            new Employee { Id = 5, Name = "Rose", Mobile = "0987-335668",
                Email = "rose@gmail.com", Department = "資訊部", Title = "工程師" }
        );

        modelBuilder.Entity<Register>().HasData(
            new Register { Id = 1, Name = "奚江華", Nickname = "聖殿祭司",
                Password = "myPassword*", Email = "dotnetcool@gmail.com",
                City = 4, Gender = 1, Commutermode = "1", Comment = "Nothing",
                Terms = true }
        );
    }
}

```

說明：

1. DbContext 是負責執行時期的 Entity 物件管理，包括從資料庫讀取資料、追蹤異動及資料寫入資料庫
2. DbSet 是用來查詢和儲存 Entity 個體資料，LINQ 查詢是以 DbSet 為對象，然後 LINQ 語法會被轉換成 SQL 查詢語法

step 04 在 appsettings.json 新增「CmsContext」資料庫連線設定

 appsettings.json

```

{
    ...
    "AllowedHosts": "*",
    "ConnectionStrings": {
        "CmsContext": "Server=(localdb)\\mssqllocaldb;Database=CmsDB;
            Trusted_Connection=True;MultipleActiveResultSets=true"
    }
}

```

step 05 在 DI Container 中註冊 CmsContext Program.cs

```
//取得組態中資料庫連線設定
string connectionString = builder.Configuration.GetConnectionString
("CmsContext");

//註冊 EF Core 的 CmsContext
builder.Services.AddDbContext<CmsContext>(options => options.UseSqlServer
(connectionString));
```

1. 從組態中讀取資料庫連線

2. 註冊 CmsContext

3. 使用 SQL Server 提供者

說明：在 DI Container 中註冊 CmsContext 後，才能在 Controller 或其他程式中以 DI 注入服務。而 options 設定了：❶使用 SQL Server Provider 提供者，❷使用 CmsContext 資料庫連線

step 06 在【套件管理器主控台】新增 Migration。**Add-Migration** InitialDB

說明：

1. 在 Migrations 資料夾會產生日期流水號「20221214085148_InitialDB」的異動檔。而 InitialDB 代表此次異動的名稱，亦可換成其他有意義的名稱
2. Add-Migration 指令需要 CmsContext 在 DI Container 中註冊，否則無法正確執行

step 07 執行「Update-Database」命令更新資料庫，第一次會建立資料庫及資料表定義，倘若 CmsContext 的 OnModelCreating 方法中有建立樣本資料，則會將樣本資料 Insert 到資料表



圖 11-4 執行 Migrations 命令

11-6-2 從 GET 與 POST 角度解釋 CRUD 四類 Views 與 Actions 的對應關係

有了資料庫樣本資料後，便可著手製作 CRUD 四類 Actions 與 Views，但在這之前，先從 GET 與 POST 角度解釋它們之間的對應關係。

❖ GET vs. POST 方法

GET 和 POST 是 HTTP 規範定義方法（Methods），功用為：

- GET：使用者在瀏覽器網址列輸入 URL 位址後，就會用 GET 方法從伺服器讀取資料。GET 方法主要是作讀取資料的動作
- POST：前端 HTML Form 在 Submit 後，會以 POST 方法向後端伺服器提交資料，意謂著有資料要回寫到伺服器，因此有時可把 POST 看成是寫入動作，包括新增、更新或刪除資料都可用 POST 來執行

在前一及下一範例中，使用 Scaffolding 產出的 CRUD 樣板，其中 Views 與 Actions 的對應關係如下表。